



Section 5 / 24

Go to Questions ↓

# Registers, Addresses, and Data Types

elements of assembly

Now that we understand general computer and processor architecture, we need to understand a few assembly elements before we start learning Assembly: **Registers**, **Memory Addresses**, **Address Endianness**, and **Data Types**. Each of these elements is important, and properly understanding them will help us avoid issues and hours of troubleshooting while writing and debugging assembly code.

## Registers

As previously mentioned, each CPU core has a set of registers. The registers are the fastest components in any computer, as they are built within the CPU core. However, registers are very limited in size and can only hold a few bytes of data at a time. There are many registers in the x86 architecture, but we will only focus on the ones necessary for learning basic Assembly and essential for future binary exploitation.

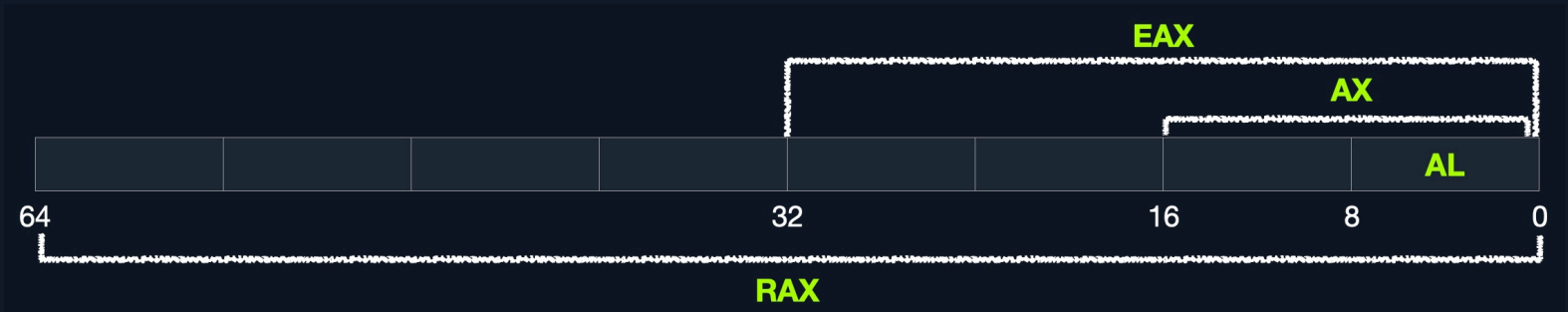
There are two main types of registers we will be focusing on: **Data Registers** and **Pointer Registers**.

| Data Registers | Pointer Registers |
|----------------|-------------------|
| rax            | rbp               |
| rbx            | rsp               |
| rcx            | rip               |
| rdx            |                   |
| r8             |                   |
| r9             |                   |
| r10            |                   |

- **Data Registers** - are usually used for storing instructions/syscall arguments. The primary data registers are: `rax` , `rbx` , `rcx` , and `rdx` . The `rdi` and `rsi` registers also exist and are usually used for the instruction `destination` and `source` operands. Then, we have secondary data registers that can be used when all previous registers are in use, which are `r8` , `r9` , and `r10` .
- **Pointer Registers** - are used to store specific important address pointers. The main pointer registers are the Base Stack Pointer `rbp` , which points to the beginning of the Stack, the Current Stack Pointer `rsp` , which points to the current location within the Stack (top of the Stack), and the Instruction Pointer `rip` , which holds the address of the next instruction.

# Sub-Registers

Each **64-bit** register can be further divided into smaller sub-registers containing the lower bits, at one byte **8-bits** , 2 bytes **16-bits** , and 4 bytes **32-bits** . Each sub-register can be used and accessed on its own, so we don't have to consume the full 64-bits if we have a smaller amount of data.



Sub-registers can be accessed as:

| Size in bits | Size in bytes | Name                                        | Example |
|--------------|---------------|---------------------------------------------|---------|
| 16-bit       | 2 bytes       | the base name                               | ax      |
| 8-bit        | 1 bytes       | base name and/or ends with <b>l</b>         | al      |
| 32-bit       | 4 bytes       | base name + starts with the <b>e</b> prefix | eax     |
| 64-bit       | 8 bytes       | base name + starts with the <b>r</b> prefix | rax     |

For example, for the **bx** data register, the 16-bit is **bx** , so the 8-bit is **bl** , the 32-bit would be **ebx** , and the 64-bit would be **rbx** . The same goes for pointer registers. If we take the base stack

pointer `bp` , its 16-bit sub-register is `bp` , so the 8-bit is `bp1` , the 32-bit is `ebp` , and the 64-bit is `rbp` .

The following are the names of the sub-registers for all of the essential registers in an x86\_64 architecture:

| Description                   | 64-bit Register  | 32-bit Register  | 16-bit Register  | 8-bit Register   |
|-------------------------------|------------------|------------------|------------------|------------------|
| Data/Arguments Registers      |                  |                  |                  |                  |
| Syscall Number/Return value   | <code>rax</code> | <code>eax</code> | <code>ax</code>  | <code>al</code>  |
| Callee Saved                  | <code>rbx</code> | <code>ebx</code> | <code>bx</code>  | <code>bl</code>  |
| 1st arg - Destination operand | <code>rdi</code> | <code>edi</code> | <code>di</code>  | <code>dil</code> |
| 2nd arg - Source operand      | <code>rsi</code> | <code>esi</code> | <code>si</code>  | <code>sil</code> |
| 3rd arg                       | <code>rdx</code> | <code>edx</code> | <code>dx</code>  | <code>dl</code>  |
| 4th arg - Loop counter        | <code>rcx</code> | <code>ecx</code> | <code>cx</code>  | <code>cl</code>  |
| 5th arg                       | <code>r8</code>  | <code>r8d</code> | <code>r8w</code> | <code>r8b</code> |
| 6th arg                       | <code>r9</code>  | <code>r9d</code> | <code>r9w</code> | <code>r9b</code> |
| Pointer Registers             |                  |                  |                  |                  |

| Description                     | 64-bit Register | 32-bit Register | 16-bit Register | 8-bit Register |
|---------------------------------|-----------------|-----------------|-----------------|----------------|
| Base Stack Pointer              | rbp             | ebp             | bp              | bpl            |
| Current/Top Stack Pointer       | rsp             | esp             | sp              | spl            |
| Instruction Pointer 'call only' | rip             | eip             | ip              | ipl            |

As we go through the module, we'll discuss how to use each of these registers.

There are other various registers, but we will not cover them in this module, as they are not needed for basic Assembly usage. As an example, there's the **RFLAGS** register, which is used to maintain various flags used by the CPU, like the zero flag **ZF**, which is used for conditional instructions.

---

## Memory Addresses

As previously mentioned, x86 64-bit processors have 64-bit wide addresses that range from **0x0** to **0xffffffffffffffff**, so we expect the addresses to be in this range. However, RAM is segmented into various regions, like the Stack, the heap, and other program and kernel-specific regions. Each memory region has specific **read**, **write**, **execute** permissions that specify whether we can read from it, write to it, or call an address in it.

Whenever an instruction goes through the Instruction Cycle to be executed, the first step is to fetch the instruction from the address it's located at, as previously discussed. There are several types of address fetching (i.e., addressing modes) in the x86 architecture:

| Addressing Mode | Description                                                        | Example                                               |
|-----------------|--------------------------------------------------------------------|-------------------------------------------------------|
| Immediate       | The value is given within the instruction                          | <code>add 2</code>                                    |
| Register        | The register name that holds the value is given in the instruction | <code>add rax</code>                                  |
| Direct          | The direct full address is given in the instruction                | <code>call 0xfffffffffaa8a25ff</code>                 |
| Indirect        | A reference pointer is given in the instruction                    | <code>call 0x44d000</code> or <code>call [rax]</code> |
| Stack           | Address is on top of the stack                                     | <code>add rsp</code>                                  |

In the above table, lower is slower. The less immediate the value is, the slower it is to fetch it.

Even though speed isn't our biggest concern when learning basic Assembly, we should understand where and how each address is located. Having this understanding will help us in future binary exploitation, with Buffer Overflow exploits, for example. The same understanding will have an even more significant implication with advanced binary exploitation, like ROP or Heap exploitation.

---

## Address Endianness

An address endianness is the order of its bytes in which they are stored or retrieved from memory. There are two types of endianness: `Little-Endian` and `Big-Endian`. With Little-Endian

processors, the little-end byte of the address is filled/retrieved first **right-to-left**, while with Big-Endian processors, the big-end byte is filled/retrieved first **left-to-right**.

For example, if we have the address **0x0011223344556677** to be stored in memory, little-endian processors would store the **0x00** byte on the right-most bytes, and then the **0x11** byte would be filled after it, so it becomes **0x1100**, and then the **0x22** byte, so it becomes **0x221100**, and so on. Once all bytes are in place, they would look like **0x7766554433221100**, which is the reverse of the original value. Of course, when retrieving the value back, the processor will also use little-endian retrieval, so the value retrieved would be the same as the original value.

Another example that shows how this can affect the stored values is binary. For example, if we had the 2-byte integer **426**, its binary representation is **00000001 10101010**. The order in which these two bytes are stored would change its value. For example, if we stored it in reverse as **10101010 00000001**, its value becomes **43521**.

The big-endian processors would store these bytes as **00000001 10101010** **left-to-right**, while little-endian processors store them as **10101010 00000001** **right-to-left**. When retrieving the value, the processor has to use the same endianness used when storing them, or it will get the wrong value. This indicates that the order in which the bytes are stored/retrieved makes a big difference.

The following table demonstrates how endianness works:

| Address       | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | Address Value |
|---------------|---|---|---|---|---|---|---|---|---------------|
| Little Endian |   |   |   |   |   |   |   |   | 0x            |
| Big Endian    |   |   |   |   |   |   |   |   | 0x            |

Address: 0x0011223344556677

Load Address

| Address       | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | Address Value |
|---------------|---|---|---|---|---|---|---|---|---------------|
| Little Endian |   |   |   |   |   |   |   |   | 0x            |
| Big Endian    |   |   |   |   |   |   |   |   | 0x            |

Address: 0x0011223344556677

Load Address

You can click on the "Load Address" button to visualize how each endianness loads data/addresses into memory.

As we can see, this means that an address written in little-endian or in big-endian would refer to different locations in the memory, as it would be read differently by each processor type.



This module will always use little-endian byte order, as it is used with Intel/AMD x86 in most modern operating systems, so the shellcode is always represented right-to-left.

The important thing we need to take from this is knowing that our bytes are stored into memory from right-to-left. So, if we were to push an address or a string with Assembly, we would have to push it in reverse. For example, if we want to store the word `Hello`, we would push its bytes in reverse: `o`, `l`, `l`, `e`, and finally `H`.

This may seem a bit counter-intuitive since most people are used to reading from left-to-right. However, this has multiple advantages when processing data, like being able to **retrieve a sub-register without having to go through the entire register** or being able to **perform arithmetic in the correct order right-to-left** such as in + we do from the units place to the tens place.

# Data Types

Finally, the x86 architecture supports many types of data sizes, which can be used with various instructions. The following are the most common data types we will be using with instructions:

| Component           | Length            | Example           |
|---------------------|-------------------|-------------------|
| byte                | 8 bits            | 0xab              |
| word                | 16 bits - 2 bytes | 0xabcd            |
| double word (dword) | 32 bits - 4 bytes | 0abcdef12         |
| quad word (qword)   | 64 bits - 8 bytes | 0abcdef1234567890 |

Whenever we use a variable with a certain data type or use a data type with an instruction, both operands should be of the same size.

For example, we can't use a variable defined as `byte` with `rax`, as `rax` has a size of 8 bytes. In this case, we would have to use `al`, which has the same size of 1 byte. The following table shows the appropriate data type for each sub-register:

| Sub-register     | Data Type          |
|------------------|--------------------|
| <code>al</code>  | <code>byte</code>  |
| <code>ax</code>  | <code>word</code>  |
| <code>eax</code> | <code>dword</code> |
| <code>rax</code> | <code>qword</code> |

We will discuss this further in the upcoming sections. With all fundamentals of Assembly covered, we can start learning about x86 assembly instructions and writing basic Assembly code.



Enable step-by-step solutions ⓘ

PRO

### Optional Exercise 1

XP +40 ^

What is the 8-bit register for 'rdi'?

Write your answer

 Submit

 Reveal Answer

[← Previous](#)

Section 5 / 24

 +20

[Next](#)

[Mark Complete & Next](#)